Introduction à la Théorie des Graphes et à ses applications

Revel-Hidalgo Cyrian
El Ouali Rayan

Déclaration sur l'honneur

Nous déclarons sur l'honneur ne pas avoir commis de plagiat ou toute autre forme de fraude et de n'avoir pas utilisé les outils d'intelligences artificielles.

Table des matières

Introduction **3**

1 Définitions **5**
1.1 Qu'est-ce qu'un graphe ? . . . 5
1.2 Graphe orienté, non orienté . . . 5
1.3 Boucle, arête multiple . . . 5
1.4 Graphe simple . . . 6
1.5 Graphe complet . . . 6
1.6 Degré . . . 7
1.7 Connexité . . . 7
1.8 Graphe biparti . . . 8

2 Les 7 Ponts de Königsberg **10**
2.1 Introduction . . . 10
2.2 Formulation du problème . . . 10
2.3 Théorème d'Euler . . . 11
2.4 Démonstration . . . 11

3 Transport de marchandise et Optimisation de Flot **14**
3.1 Introduction . . . 14
3.2 Réseau de flot . . . 14
3.3 Algorithme de Ford-Fulkerson . . . 14
3.4 Coupe minimale . . . 17

4 Optimisation d'un algorithme par un graphe **18**
4.1 Introduction . . . 18
4.2 Présentation du problème . . . 18
4.3 Modélisation de la situation initiale . . . 19

5 Approche algorithmique **22**
5.1 Optimisation des recherches . . . 22
5.2 Algorithme du minimax . . . 22
5.3 Élagage Alpha-Beta . . . 28

Conclusion **30**

Introduction

L'histoire de la théorie des graphes débute au 18e siècle avec les travaux d'Euler. Celui-ci s'intéresse à l'étude de divers problèmes tels que celui des ponts de Königsberg, la marche du cavalier sur l'échiquier ou encore le problème du coloriage de cartes et du plus court trajet entre deux points.



FIGURE 1 – Carte historique de Königsberg et Leonhard Euler

Le terme de "graphes" est lui introduit en 1822 par James Joseph Sylvester, un mathématicien et géomètre Anglais.

La force de la théorie des graphes est qu'elle est multidisciplinaire, on la retrouve en biologie, en chimie, en sciences sociales, en informatique...

Depuis le début du 20e siècle, grâce aux travaux de mathématiciens tels que König, Cayley, Menger puis de Berge et d'Erdös, la théorie des graphes est considérée comme une branche à part entière des mathématiques.

De manière générale, un graphe permet de représenter la structure d'un ensemble en exprimant les relations entre ses éléments : réseau routier, réseau de communication, réseau de neurones, interaction de diverses espèces animales...

De nos jours, ce sont les informaticiens qui contribuent majoritairement aux derniers travaux du fait de l'importance de l'aspect algorithmique en théorie des graphes.

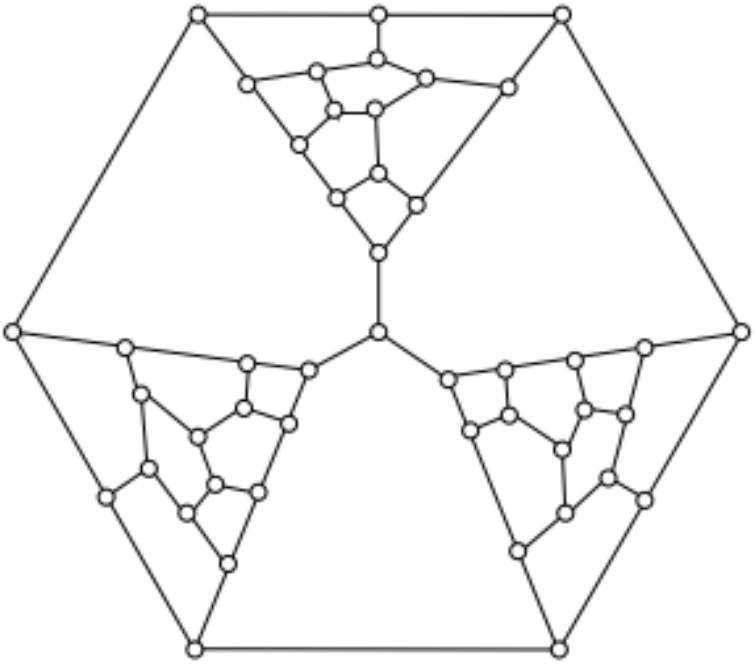


FIGURE 2 – Graphe de Tutte

1. Définitions

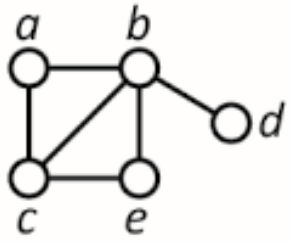
1.1 Qu'est-ce qu'un graphe ?

En théorie des graphes, un graphe G est une paire $G = (V, E)$ d'ensembles où :
— $V = V(G)$ est l'ensemble (non-vide) des sommets du graphe G, et
— $E = E(G)$ est l'ensemble des arêtes du graphe G, où chaque arête est une paire de sommets.

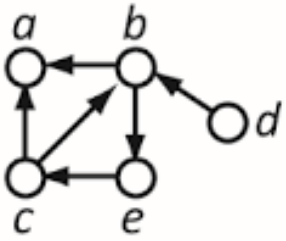
1.2 Graphe orienté, non orienté

Dans le cas d'un graphe non-orienté, les arêtes relient 2 sommets "à double sens", ainsi l'arête (a, b) qui relie le sommet a et le sommet b est la même que l'arête (b, a).

Pour un graphe orienté, l'ordre importe car ses liaisons sont à sens uniques. On parlera alors d'arc, avec la notation (a, b) un arc sortant du sommet a et entrant au sommet b.



un graphe non orienté



un graphe orienté

1.3 Boucle, arête multiple

Une arête reliant un sommet à lui-même est une boucle. Sur le graphe de gauche de la figure 1.1, le sommet 2 possède une boucle.

Plusieurs arêtes sont dites parallèles si elles relient un même couple de sommets. On dit alors que c'est une arête multiple. Le graphe de droite de la figure 1.1 possède une arête multiple entre les sommets 2 et 4.

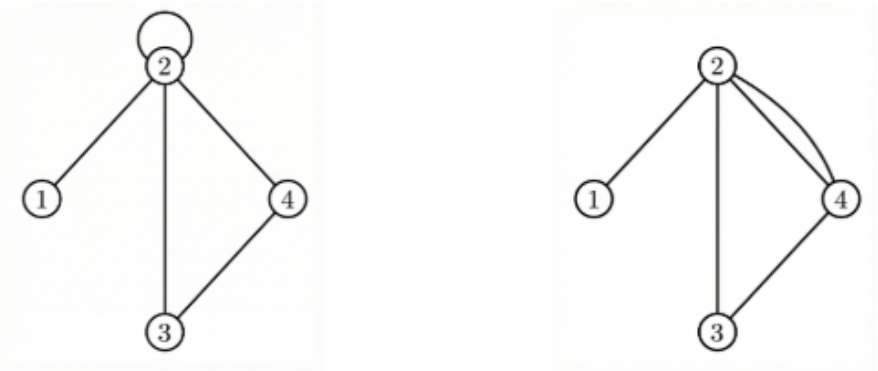


FIGURE 1.1 – Exemple de boucle (à gauche) et d'arête multiple (à droite)

1.4 Graphe simple

On appelle graphe simple un graphe qui ne possède ni boucle, ni arête multiple. Les graphes des figures 1 et 2 ne sont donc pas simples. Le graphe non orienté de la page précédente, lui, est simple.

1.5 Graphe complet

Un graphe simple dont chaque sommet est relié à tous les autres sommets par une arête est dit complet.

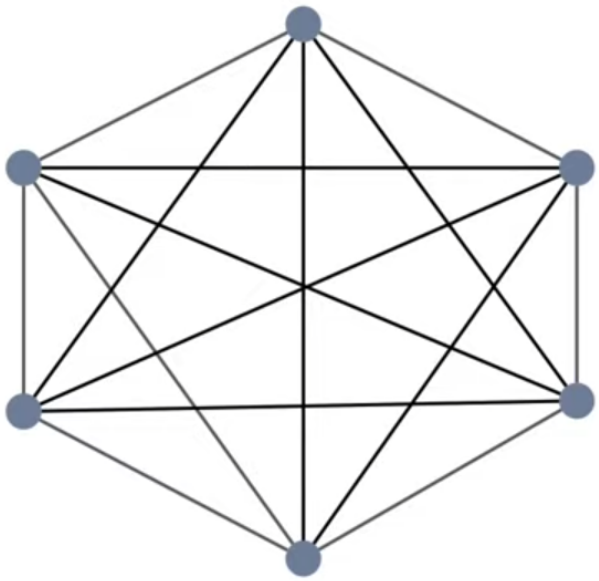


FIGURE 1.2 – Exemple de graphe complet

1.6 Degré

Dans un graphe non orienté, le degré noté $\deg(v)$ d'un sommet v est égal au nombre d'arête incidente à v (une boucle compte pour 2 arêtes). Un sommet dont le degré vaut 0 est dit isolé.

1.7 Connexité

Un graphe non orienté est dit connexe si de chaque sommet il est possible d'aller à n'importe quelle autre sommet du graphe.

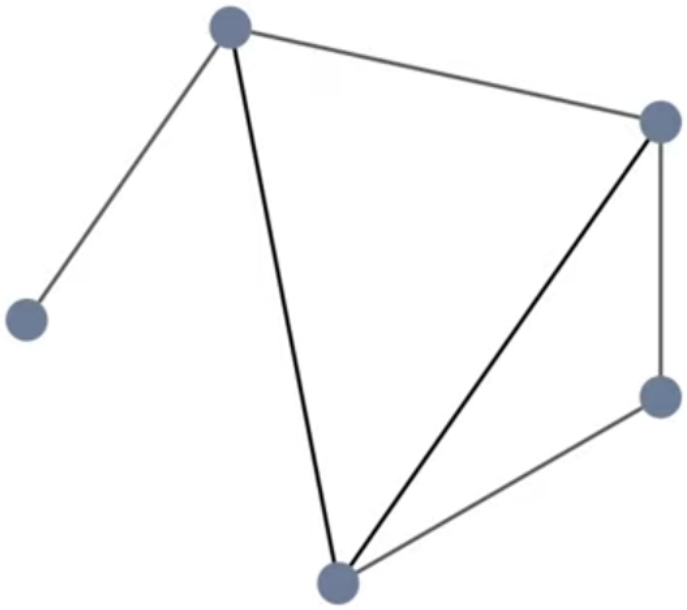


FIGURE 1.3 – Exemple de graphe connexe

Une composante connexe est un sous-graphe connexe qui ne fait partie d'aucun autre sous-graphe connexe.

Exemple : Le graphe ci-dessous n'est pas connexe, mais il possède deux composantes connexes : $\{A, D, C, F\}$ et $\{B, E\}$.

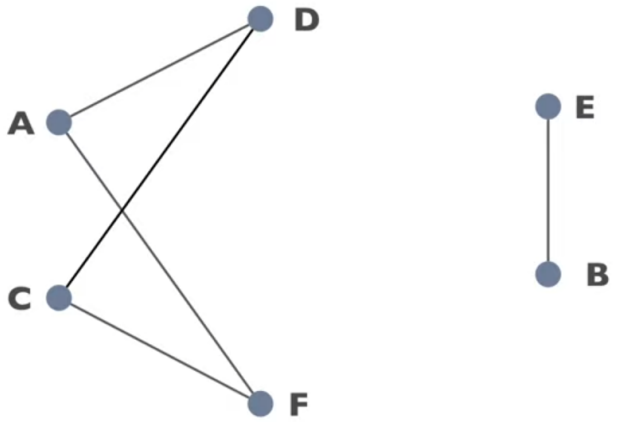


FIGURE 1.4 – Exemple de graphe non connexe

1.8 Graphe biparti

Un graphe (non-orienté) est biparti si l'on peut partager l'ensemble de ses sommets en deux sous-ensembles U et V tels que chaque arêtes relie un sommet de U et un sommet de V (c'est à dire qu'il n'existe aucune arête qui relie deux sommets appartenant à un même sous-ensemble).

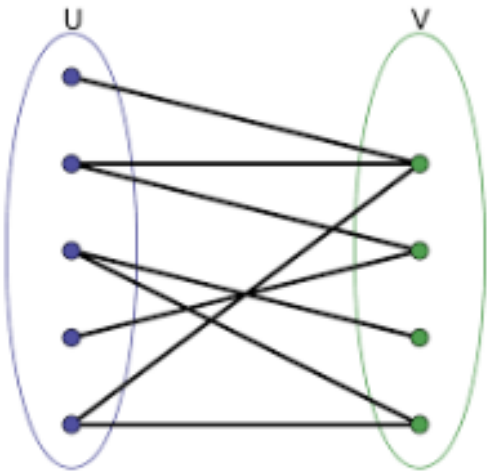


FIGURE 1.5 – Exemple de graphe biparti

Remarque : Un graphe est biparti si et seulement s'il ne contient pas de cycle de longueur impaire. (La démonstration est laissée en exercice au lecteur)

Ce type de graphe est utilisé par les plateformes de streamings dans leurs algorithmes de recommandations. Un ensemble correspond aux films et séries, et l'autre correspond aux utilisateurs. Un individu est relié à un sommet si il a regardé le film/série. Si des utilisateurs ont plusieurs voisins communs (reliés à des mêmes sommets), ils auront le même "profil", ils seront alors susceptibles d'apprécier ce que l'autre individu a regardé, ça leur sera donc recommandé par la plateforme.

2. Les 7 Ponts de Königsberg

2.1 Introduction

Königsberg est une ville située en Russie, elle est actuellement connue sous le nom de Kaliningrad. Celle-ci possède une particularité géographique : elle est traversée par le fleuve Pregel qui divise la ville en 2 parties terrestres et 2 îles, reliées par 7 ponts. Cette configuration mena les habitants à se poser une question, qui fût reprise par le mathématicien Euler : **Est-il possible de concevoir une promenade permettant de traverser chaque pont une seule et unique fois, tout en revenant au point de départ ?**

2.2 Formulation du problème

Pour résoudre ce problème, le célèbre mathématicien simplifia le problème en représentant la ville de Königsberg sous la forme d'un graphe : Chaque sommet représente un bout de terre, et chaque arête un pont.



FIGURE 2.1 – Simplification du problème

Le problème revient à se demander si le graphe obtenu est un graphe eulérien, c'est-à-dire un graphe possédant un cycle eulérien : une chaîne passant par toutes les arêtes et dont le sommet de départ est égal au sommet d'arrivée. Par exemple, le graphe suivant est eulérien car 1-2-3-4-1 est un cycle eulérien.

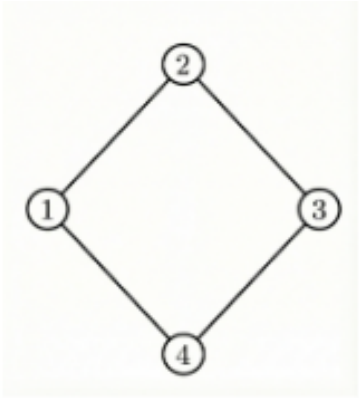


FIGURE 2.2 – Exemple de graphe eulérien

2.3 Théorème d'Euler

Euler a alors formulé les conditions pour déterminer si un graphe est ou non eulérien (ce qu'on appelle aujourd'hui le théorème d'Euler) :

1. **Le graphe doit être connexe et non orienté**
2. **Tous les sommets du graphes doivent être de degré pair**

Qu'en est-il de Königsberg ? On vérifie bien la première condition, le graphe présenté plus haut est bien non orienté, et connexe car tous ses sommets sont accessibles depuis n'importe quel autre sommet. Cependant, on remarque que tous ses sommets sont de degré impair (B,C,D de degré 3 et A de degré 5). La deuxième condition n'étant pas respectée, la réponse au problème est qu'il n'existe pas de promenade passant par tous les ponts une seule et unique fois.

2.4 Démonstration

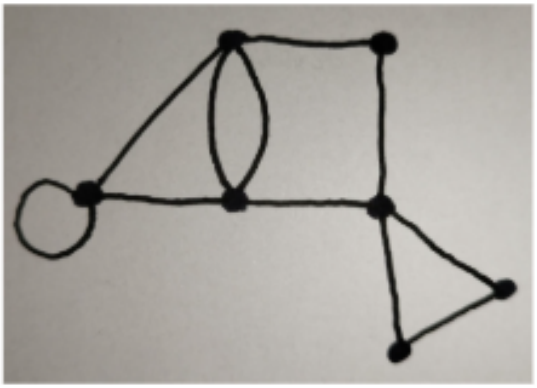
Démonstration du théorème d'Euler :

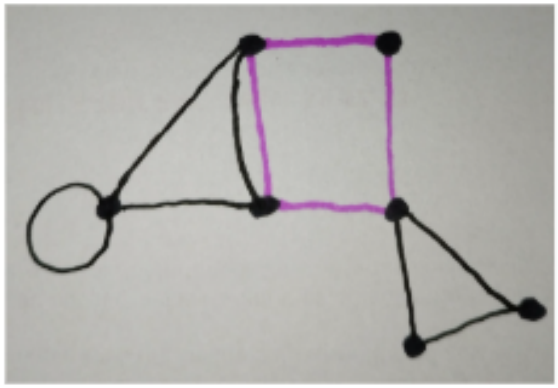
$=>$ Soit G un graphe eulérien, G est donc connexe et possède un cycle eulérien C. On choisit un sommet de départ u, et passe par chaque autre sommet en empruntant une arête pas encore utilisée, en suivant le cycle C. Pour tout sommet $v \neq u$, on a bien une arête qui entre et une qui sort. Donc à chaque fois qu'on passe par un sommet, on ajoute 2 à son degré (+1 en entrant, +1 en sortant). Comme C est un cycle, on finit par revenir au sommet u, ce qui rajoute 1 à son degré. Ainsi, chaque sommet de G a un degré pair.

$<=$ Soit G un graphe non orienté connexe, dont chaque sommet est de degré pair. Montrons que G est eulérien.

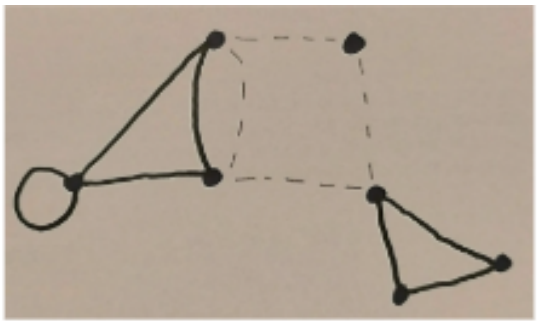
On procède par récurrence sur m, le nombre d'arête de G. Pour $m = 0$: Chaque sommet est isolé, or comme G est connexe il ne peut y avoir qu'un sommet. G est donc trivialement eulérien (en empruntant aucune arête).

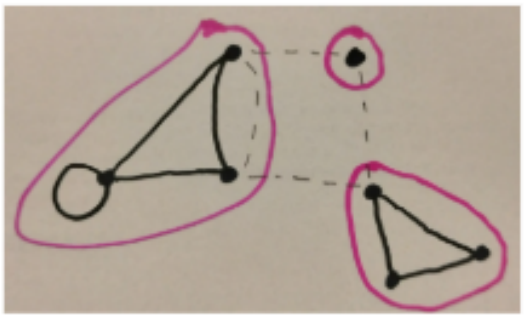
Pour $m > 0$: Supposons la propriété vraie au rang $m-1$, montrons qu'elle se prolonge pour m arêtes. Comme G est connexe, il n'y a pas de sommet isolé, donc pas de sommet de degré 0. Donc pour tout sommet v, $\deg(v) \geq 2$. On peut donc affirmer que G possède un cycle C (en rose sur l'exemple).



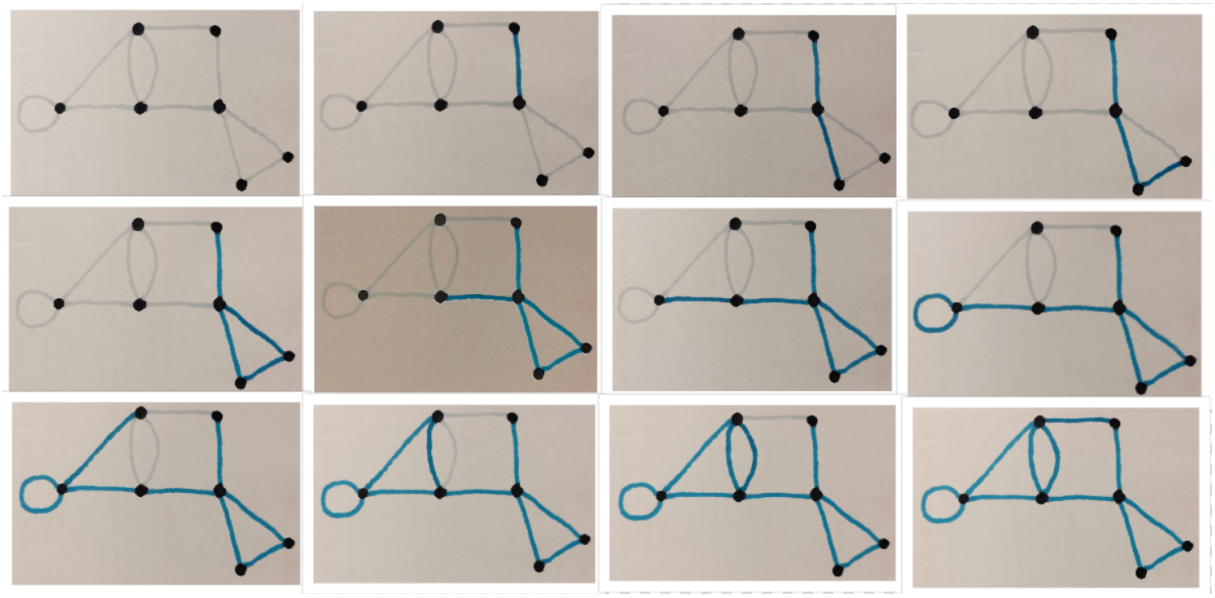


On supprime les arêtes appartenant au cycle C, ce qui conserve la parité des degrés des sommets (car pour chaque sommet dans le cycle on enlève une arête qui entre et une qui sort). On a alors obtenu de nouveaux graphes dont les sommets sont tous de degré pair et dont le nombre d'arête est strictement inférieur à m.





D'après notre hypothèse de récurrence, chacunes de ces composantes connexes (entourées en rose) possèdent un cycle eulérien (pour le sous-graphe qu'il représente). On a alors mis en évidence un cycle (C) qui relie différents sommets qui sont chacuns le départ d'un cycle eulérien (au sens de la composante connexe à laquelle ils appartiennent lorsque les arêtes de C sont supprimées). Donc, G possède un cycle eulérien qu'on peut retrouver en parcourant C, pas à pas : à chaque sommet de C, on dévie de C pour passer dans le sous-cycle eulérien de la "composante connexe".



Ainsi, G est eulérien.

□

3. Transport de marchandise et Optimisation de Flot

3.1 Introduction

Imaginons qu'on soit mandaté par une entreprise afin de transférer le maximum de produits en même temps d'un entrepôt à un autre. On dispose d'un plan, indiquant nos centres S et T, et entre ces points, différents entrepôts ainsi que les voies de chemins de fer qui relient ces derniers entre eux. Chaque voie est à sens unique, et possède une capacité qui correspond au nombre de marchandises qui peuvent transiter par cette voie à un même instant.

Notre objectif est de déterminer combien de marchandises peuvent être transportées en même temps, et connaître la répartition optimale sur les différentes voies.

C'est ce que l'on appelle un problème d'optimisation de flot, qui correspond à de nombreuses situations réelles (distribution d'électricité, transport de marchandise, évacuation d'eau..) On rapporte même que durant la Guerre Froide, l'US Air Force aurait utilisé le problème de flot maximum afin d'impacter au maximum le réseau ferré de l'Union Soviétique.

3.2 Réseau de flot

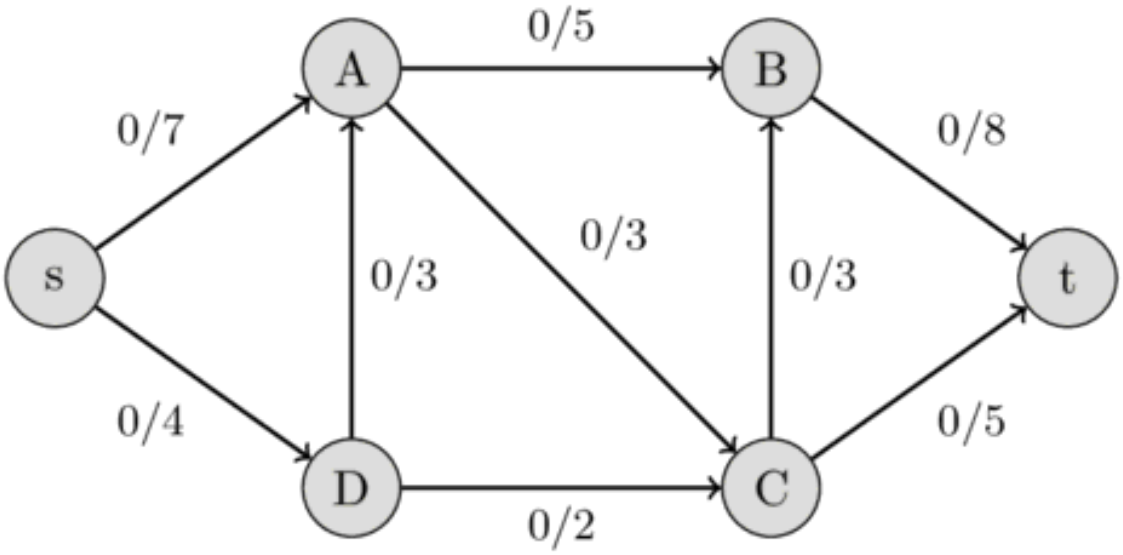
Un réseau de flot $(G, s, t, c())$ est un graphe orienté G, pour lequel on spécifie deux sommets spéciaux : la source s (sommet sans arc entrant) et un sommet de destination t (le puits, sans arc sortant) ; chaque arc e est muni d'une capacité $c(e)$. Le flot sur un réseau de flot est une fonction f qui attribue une quantité de flot à chaque arc de G en respectant les règles suivantes :

- **le flot d'un arc ne doit pas dépasser sa capacité :** $f(e) \leq c(e)$
- **pour tout sommet différent de** s **et** t**, la quantité de flot entrant doit être égale à la quantité de flot sortant.**

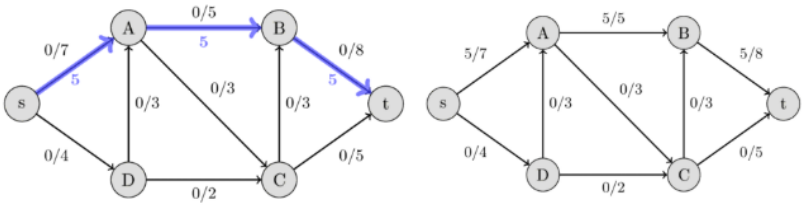
3.3 Algorithme de Ford-Fulkerson

Au fil du temps, de nombreux algorithmes ont été conçus afin de déterminer le flot optimal d'un réseau. Nous allons nous intéresser à celui de Ford-Fulkerson qui a été créé dans les années 1950.

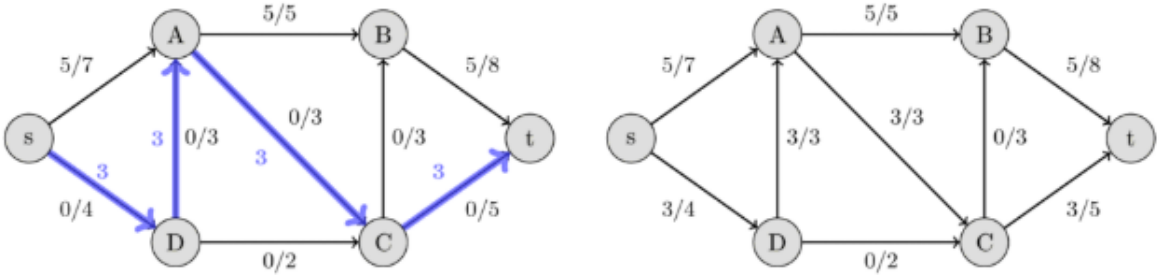
On part d'un réseau dont le flot initial est nul.



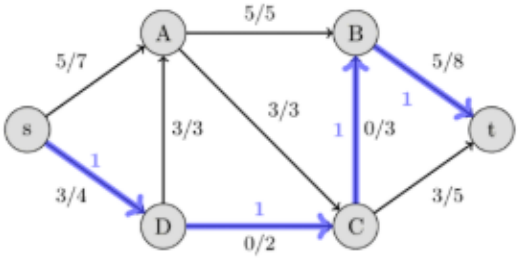
On cherche ensuite une chaîne améliorante, c'est-à-dire un chemin par lequel il est possible d'augmenter le flot sur chaque arête. Prenons $s - A - B - t$, les capacités des arcs sont 7, 8 et 5, il faut alors prendre le minimum et mettre à jour le flot.

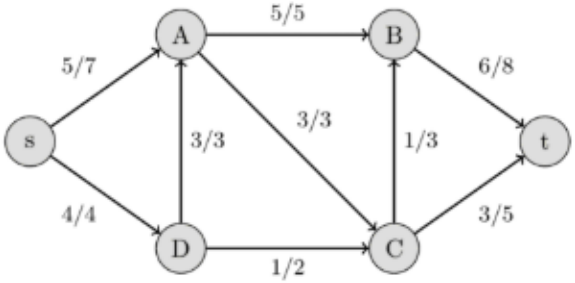


A nouveau, on trouve une chaîne améliorante, $s - D - A - C - t$ dont la capacité la plus faible est 3.

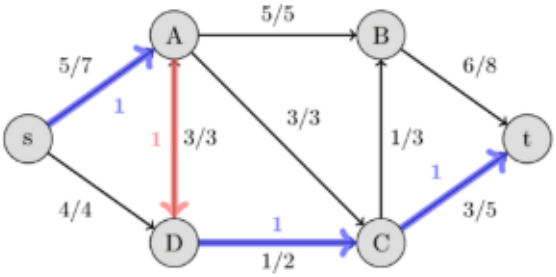


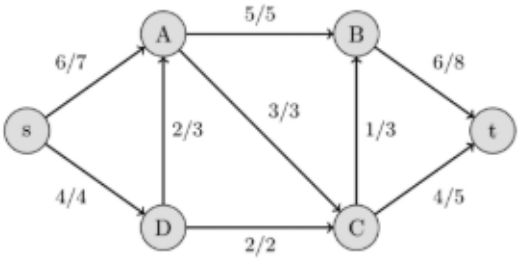
Puis, la chaîne $s - D - C - B - t$, pour rajouter un flot total de 1.





A présent, nous ne pouvons plus simplement augmenter le flot pour trouver une chaîne améliorante. Il faut redistribuer le flot et envoyer 1 de plus sur l'arc (D, C). Mais pour cela il est nécessaire de compenser en augmentant de 1 le flot de (s, A) afin de conserver les propriétés du réseau de flot. La chaîne améliorante est donc $s - A - D - C - t$.



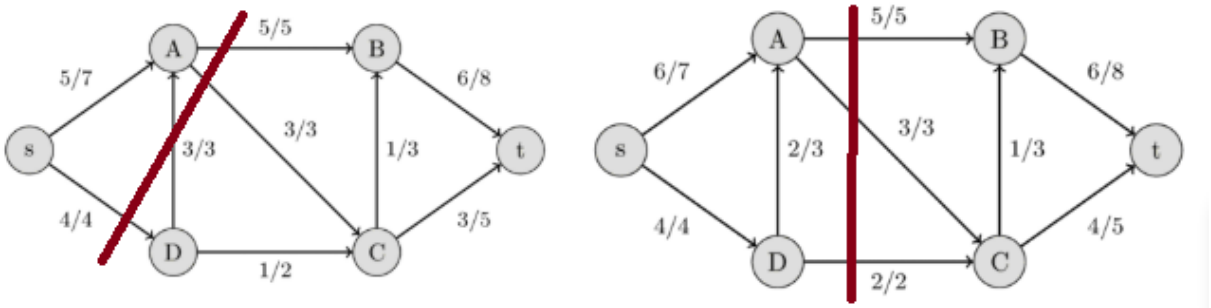


Ainsi, on obtient un flot maximal de 10, ce qui correspond bien à la fois au flot sortant de s et aux flot entrant de t.

3.4 Coupe minimale

Une manière de vérifier que l'on ne peut plus trouver de chaîne améliorante, et donc que le flot trouvé est la solution optimale, consiste à trouver une coupe minimale. Une coupe est une partition (S, T) des sommets de G. La capacité d'une coupe $c(S, T)$ est définie comme la somme des capacités de tous les arcs reliant un sommet de S à un sommet de T.

Théorème : Un flot f est maximal si et seulement il existe une coupe (S, T) telle que $f = c(S, T)$.



Ce théorème explique pourquoi l'algorithme ne s'arrête pas à l'étape de gauche ($f = 9$ et $c(S, T) = 15$) et confirme que le flot obtenu à droite est optimal ($f = c(S, T) = 10$).

4. Optimisation d'un algorithme par un graphe

4.1 Introduction

Dans cette seconde partie, nous allons nous donner à résoudre un problème dans une situation particulière. Dans un jeu d'échecs, nous cherchons à comprendre les fondements d'une approche par intelligence artificielle afin d'optimiser et comprendre les coûts de chaque position dans le jeu. Nous allons nous intéresser à modéliser cette situation par un graphe pondéré, en marquant chaque coup légal par un point, et chaque valeur du coup joué par une arête. On peut alors se demander quelle suite d'arête sera la plus efficace pour augmenter nos chances de gagner.

4.2 Présentation du problème

Contexte historique : Dans les années 1990, dans une entreprise du nom de IBM, c'est un dénommé Feng-hsiung Hsu qui créera la première intelligence artificielle capable de battre "tous les humains", en mettant "un terme" à la domination du légendaire Garry Kasparov en 1997. Étudions d'abord l'intérêt et le fonctionnement d'une telle machine pour comprendre comment prendre en compte les différents éléments d'une telle bataille.



FIGURE 4.1 – Kasparov joue une partie contre DeepBlue de Feng-hsiung Hsu (IBM)

On modélise d'abord la situation initiale d'un jeu d'échecs pour donner un exemple : Pour les données suivantes, nous allons nous baser sur une intelligence artificielle célèbre nommée "Stockfish" (et de sa version 17.1) qui nous aidera à modéliser la pondération du graphe : expliquons comment nous allons procéder pour celle-ci : Dans ce qui suit. Nous nous donnerons à jouer les blancs dans le jeu, nous commencerons donc la partie.



FIGURE 4.2 – Intelligence artificielle stockfish, open source, n°2 mondial dans le classement chess engine.

Lors d'un jeu initial, les blancs ont 18 coups possibles d'ouverture, avancer un pion regroupe 16 coups (avancer de 1, ou avancer de 2 pour chacun des 8 pions) puis faire avancer un de ses deux cavaliers.

4.3 Modélisation de la situation initiale

Notation : on note par la suite chaque coup jouable par un sommet, le nom du sommet représente le coup joué. par exemple avancer un cavalier en f3 est désigné par le coup : Cf3, avec l'initial de la pièce jouée en première position du nom suivi de la position où elle est amenée. De la même manière, on désigne un fou, par la lettre F, une tour par la lettre T etc.. (il n'y a aucun doublon, donc pas d'ambiguïté)

A noter que les pions n'ont pas de nom spécifiques, par exemple, avancer un pion en e4 est désigné simplement par le coup : "e4".

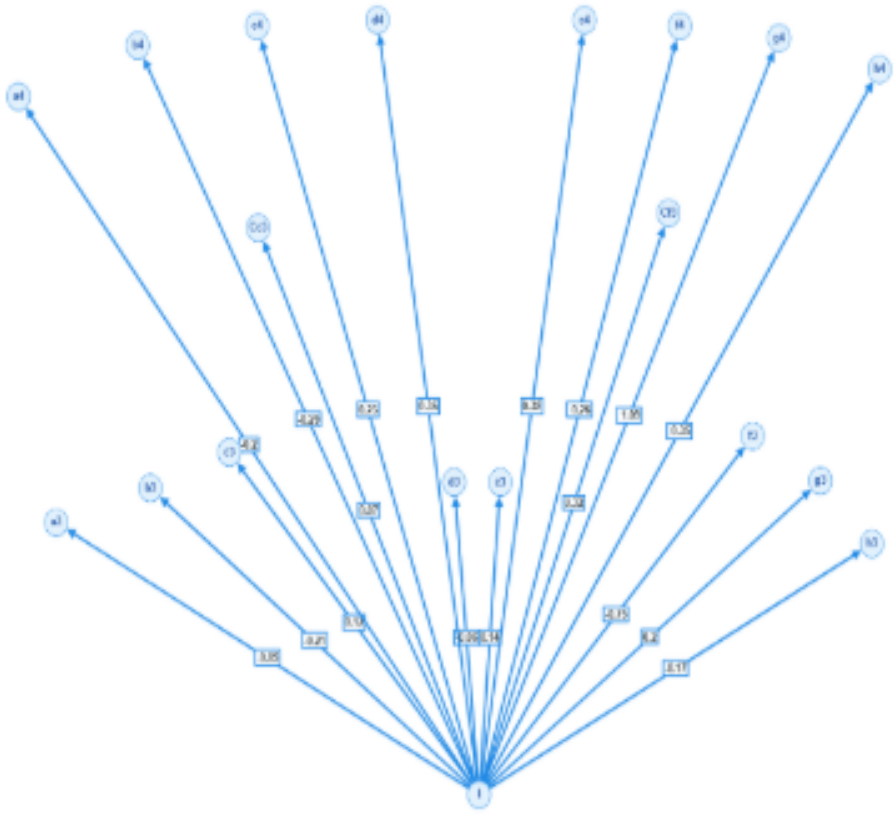
Ensuite, "manger" une pièce est désignée par l'abréviation ("Pièce qui mange" x "position de la pièce mangée"). Par exemple, si un fou mange une pièce qui se situe en f4, on notera : Fxe4.

Dernière précision (la plus subtile) lorsqu'un pion mange une pièce, on note la colonne du pion mangeur. Par exemple, si on a un pion en f4 qui mange un pion en e5, on notera "fxd5" à ne pas confondre avec "Fxd5" qui désigne un fou qui a mangé une pièce en d5. Ici, l'utilisation des majuscules est importante et désigne une pièce majeure (autre qu'un pion).



FIGURE 4.3 – Echiquier en position de départ

Le graphe associé est un graphe orienté, pondéré par la valeur d'utilité.



On désignera la valeur d'utilité par l'intérêt porté par l'intelligence artificielle à un coup donné. plus cette valeur est élevée, plus le coup est un "bon" coup. Dans la situation initiale, on peut ainsi montrer que "g4" est le moins bon, car il possède une valeur d'utilité de -1.09, tandis que "e4" est le meilleur coup avec une valeur d'utilité à 0.38. A noter que dans cette disposition, une valeur positive signifie que le coup procure un avantage dans la partie, une valeur négative signifie que le coup donne un avantage à l'adversaire.

On a ainsi un graphe que l'on note G, orienté et pondéré. et on note l'ensemble des positions légales du jeu. Soit : $G = (V, E)$ où :

— V est l'ensemble des positions légales du jeu

— $(u, v) \in E$ si et seulement s'il existe un coup légal permettant de passer de la position u à la position v.

Pour toute position $v \in V$ du jeu, le nombre de positions accessibles depuis v en un coup est fini :

— Le nombre de pièces est fini

— chaque pièce possède un nombre fini de coups légaux

— les règles du jeu interdisent les coups illégaux

Donc l'ensemble des coups depuis une position donnée est finie, ce qui implique que le degré sortant de chaque sommet est fini.

Pour le cœur de la procédure utilisée par l'intelligence artificielle, nous allons parler de tout ce qui est relatif à l'algorithmique, l'élagage beta-alpha, l'approche du min-max, et à la façon dont les problèmes d'optimisation selon la totalité des possibilités intéressantes d'un jeu d'échecs sont couvertes. Nous allons nous intéresser dans cette partie à la façon dont un graphe peut être optimisé selon une approche algorithmique.

5. Approche algorithmique

5.1 Optimisation des recherches

Pour commencer, il est bon de savoir pourquoi nous sommes confrontés à un problème d'optimisation, en effet, sur un plateau d'échecs, nous avons un nombre explosif de possibilités de jeu. Récemment, le nombre de parties de jeu éventuelles a été estimé à environ 10^{120} parties possibles. Il est évident qu'aucun ordinateur ne pourrait toutes les calculer. Il nous faut donc une approche différente, nous ne pourrons pas utiliser un algorithme "brut-force" qui utilisera une méthode exhaustive mais un algorithme plus malin, qui doit utiliser des principes plus complexes.

Utiliser une valeur d'utilité :

Pour faire des choix logiques, il faut que l'ordinateur puisse utiliser une valeur telle qu'à chaque coup, on associe une valeur plus ou moins grande selon qu'il soit favorable ou non pour le joueur. Il nous suffit donc d'utiliser la maximisation sur G pour chaque position.

On a la définition suivante : Soit vn la position courante, le meilleur coup $vn + 1$ est donné par :

$$vn + 1 = \max f(v), v \in \Gamma + (v0)$$

- $\Gamma+$ est l'ensemble des coups légaux, on note ainsi $\Gamma+(vn)$ l'ensemble des coups légaux accessibles depuis une position vn. - f est la fonction de calcul de la valeur d'utilité (propre à chaque IA), on la note ainsi et on n'essaiera pas de la décomposer dans le contenu de ce mémoire.

Ainsi, le coup optimal est celui qui amène à une position dont l'évaluation est maximale.

Concentrons nous désormais sur le cœur de la problématique de l'optimisation d'un tel graphe.

5.2 Algorithme du minimax

Le cœur de la démonstration de ce mémoire est lié à l'algorithme du minimax, il s'agit d'un algorithme pour les jeux à deux joueurs à somme nulle. Un jeu à somme nulle est un jeu où la somme des gains de l'un constitue une perte pour l'autre, dans notre exemple, si les blancs prennent l'avantage, alors les noirs sont désavantagés.

Soit $G = (V, E)$ un graphe orienté fin représentant un jeu à deux joueurs.

— Les sommets de profondeur paire correspondent aux coups joués par les blancs
— Les sommets de profondeur impaire sont les coups joués par les noirs
— Les feuilles sont évaluées par la fonction f (fonction de calcul de la valeur d'utilité)

Comme les variantes au jeu d'échecs sont totalement imprévisibles et extrêmement vaste, dans cette définition du minimax, on va se donner à prendre pour exemple une finale de partie d'échecs très simple qui est la suivante :

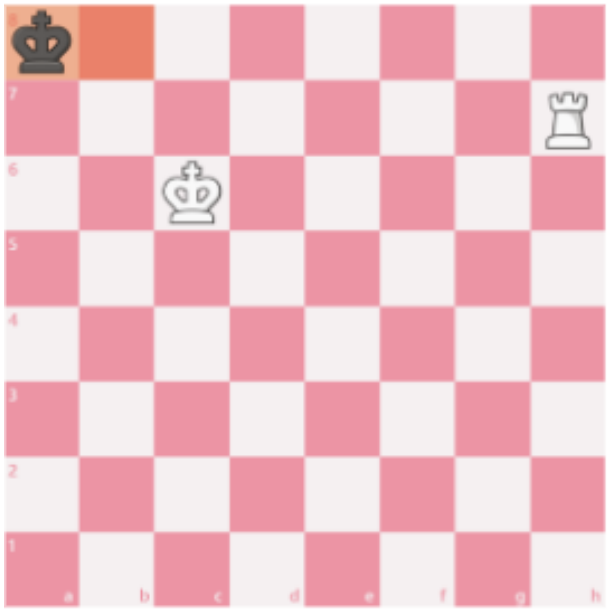


FIGURE 5.1 – Finale simple avec un mat en 2 coups pour les blancs

La fonction du minimax prends en paramètre :
— Les ensemble de noeuds
— L'ensemble des arêtes
— la position initiale (que l'on notera ici $n0$)

On garde toujours la fonction f étant la fonction d'évaluation de la valeur d'utilité

La valeur d'utilité est une valeur réelle, s'il y a un gagnant, on notera la valeur comme $+\infty$ si ce sont les blancs qui gagnent la partie, sinon on écrira $-\infty$ si ce sont les noirs qui gagnent la partie. Par ailleurs on note MX la possibilité pour le joueur de faire un Mat (gagner la partie) avec la notation M suivie de "X", X étant le nombre de coups maximum requis dans le jeu optimal des deux adversaires. Par exemple, M3 signifie que dans cette position donnée, le joueur peut finir la partie en 3 coups si les deux joueurs jouent parfaitement.

On note un avantage de Mat en 3 coups :
— M3 pour les blancs
— -M3 pour les noirs

Voici la modélisation de cette position en un graphe :

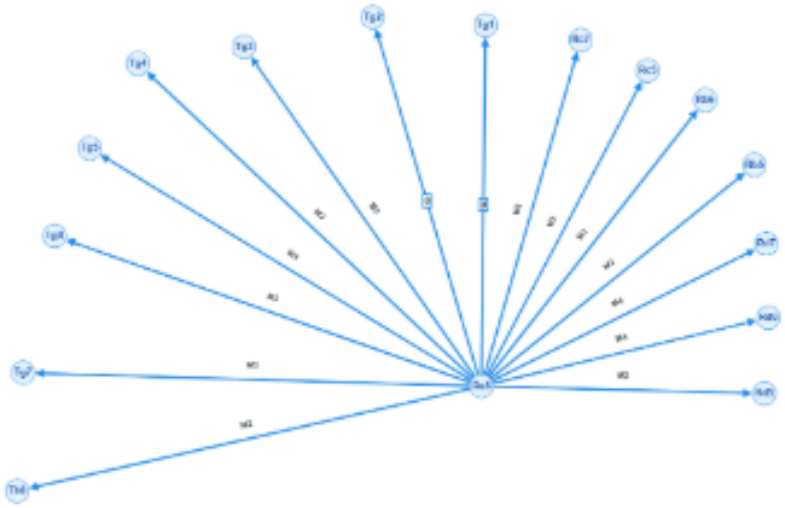
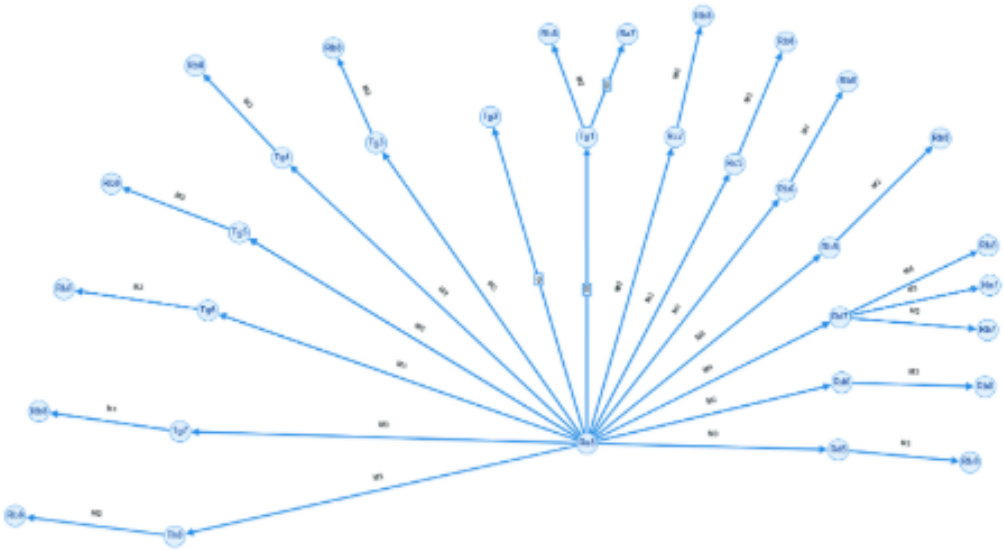


FIGURE 5.2 – Ensemble des coups pour les blancs

On peut ainsi ajouter à chaque coup, les coup suivant qui sera calculé pour les noirs :



On voit qu'une simple position avec trois pièces peut déjà constituer un ensemble assez vaste de possibilités : Par ailleurs on notera que le coup Tg2 met immédiatement fin à la partie en faisant un jeu nul.

Principe du minimax, l'intelligence artificielle possède une profondeur de calcul, (en fait c'est le nombre de coups "anticipés" qu'elle peut calculer) ici, on a une profondeur de 2 dans l'exemple précédent.

Soit un état final (une feuille) nf, l'ordinateur va chercher à maximiser la valeur de la valeur d'utilité si le coup est joué par les blanc (noeud pair) ou à minimiser la valeur d'utilité du coup joué par les noirs (noeud impair).

On utilise un algorithme récursif tel que :

— Soit $d \in \mathbb{N}^*$ la profondeur restante du coup (ou bien la profondeur maximale calculée par l'ordinateur, par exemple en début de partie)

— Soit nf l'état final mettant fin à la partie, n le nœud courant et n' le nœud enfant.

Dans l'exemple précédent, partons de l'état n, qui est Ra8 (la racine).

On calcule à l'aide de la fonction f chaque coup possible pour les blancs, on a : $enfants(n0) = \{n1, n2, \ldots, ni\}$ avec chaque position associée où : $n1$ = position après Th8, $n2$ = position après Ta7 etc..

On présente la fonction minimax en pseudo-code :

```
minimax(racine, profondeur, (Min ou Max)){
    Pour tous les enfants du noeud racine {
        res <- on calcule la valeur d'utilité max ou min
               (relative à celle en entrée)
    }
    renvoyer res ;
}
```

On peut démontrer pourquoi ce code récursif fonctionne pour tous les jeux à deux joueurs à somme nulle.

Comme il s'agit d'un algorithme récursif, on se propose de le démontrer par une récurrence :

On définit le joueur par : $joueur = \{Max$ si d est pair, Min si d est impair $\}$

On définit la fonction minimax récursivement par :

$$minimax(n,d) = \begin{cases} f(n) & \text{si } d = 0 \text{ (cas initial)} \\ \max(minimax(n', d-1)) & \text{si joueur} = \text{Max} \\ \min(minimax(n', d-1)) & \text{si joueur} = \text{Min} \end{cases}$$

Récurrence : $P(n)$: Soit $n \in N$ un noeud de l'arbre de jeu et $d \in N$ une profondeur quelconque. Alors $minimax(n, d)$ représente la valeur optimale que le joueur actif en n peut garantir en jouant optimalement, en supposant que l'adversaire joue également de manière optimale.

Initialisation : Pour $P(0)$;

On veut montrer que pour $d = 0$, $minimax(n, 0) = f(n)$

Par définition, $minimax(n, 0) = f(n)$. Par ailleurs, lorsque $d = 0$ on évalue directement la position enfin, si n est terminal (position finale), $f(n)$ donne directement la valeur du jeu.

On en conclut que $P(0)$ est vraie, donc que l'initialisation est vérifiée.

Hérédité : $P(0)$ est vraie, soit $n, d \in \mathbb{N}$, supposons que $P(n)$ est vraie, et montrons que $P(n+1)$ est vraie aussi, c'est-à-dire que pour tout noeud n' et toute profondeur k', $minimax(n', d')$ retourne la valeur optimale que le joueur actif peut garantir.

Cas 1) Si $joueur(n) = Max$

Par définition, on a : $minimax(n, k+1) = minimax(n', k)$ avec n' un enfant de n. Pour chaque enfant de n, $minimax(n', k)$ donne la valeur optimale que Min peut garantir à n'.

De la même manière, la valeur optimale que Max peut garantir est exactement $\max(minimax(n', k))$.

Preuve : On note argmax l'argument qui effectue le maximum à une position donnée, par exemple, si :

— Th8 => minimax = +8
— Ta7 => minimax = +2
alors $argmax(\max(minimax(Rd8, k))) = Th8$

Borne inférieure :
Max peut garantir au moins un $\max(minimax(n', k))$, car il lui suffit de choisir un coup menant à $argmax(minimax(n', k))$. Après un tel coup, même si Min joue de façon optimale, alors Max obtient $argmax(minimax(n', k))$.

Borne supérieure : Max ne peut pas garantir plus que $\max(minimax((n', k)))$ car : Quel que soit le coup de Max menant à un enfant de n', Min jouera optimalement Le meilleur coup que max puisse faire est donc de maximiser sur tous ses choix

Conclusion du cas 1 : $minimax(n, k + 1) = \max(minimax(n', k))$ et est bel et bien la valeur optimale pour Max.

Cas 2 : $joueur(n) = Min$
Par définition : $minimax(n, k + 1) = \min(minimax(n', k))$ avec n' un enfant de n En utilisant un raisonnement similaire au cas 1, on a que : La valeur optimale que Min peut garantir est exactement $\min(minimax(n', k))$

Borne inférieure : Min peut garantir que Max n'obtienne pas plus que $\min(minimax(n', k))$ en choisissant le coup qui minimise

Borne supérieure : Min ne peut pas amener à moins que $\min(minimax(n', k))$ en choisissant le coup minimisant Conclusion du cas 2 : $minimax(n, k+1) = \min(minimax(n', k))$

Conclusion : Par le principe de récurrence, on a :
— $P(0)$ est vraie
— Pour tout $k \geq 0$, $P(k) \Rightarrow P(k + 1)$
Donc pour tout $d \in \mathbb{N}$ est vraie.

□

On peut ensuite se demander pourquoi le graphe d'état d'une partie d'échecs n'est pas réprésenté par un arbre , en effet, le jeu permet d'obtenir plusieurs positions similaires en utilisant des coups différents, on peut ainsi dire que le jeu d'échecs forme un graphe car il admet des transpositions, elles sont fusionnées lorsqu'une position similaire est trouvée, donc une même position aux échecs peut avoir plusieurs parents :

Exemple : Soit la position suivante :



FIGURE 5.3 – Position classique d'une ouverture italienne au 4e coup

Graphes associés : voici deux graphes quelconques associés à cette position :

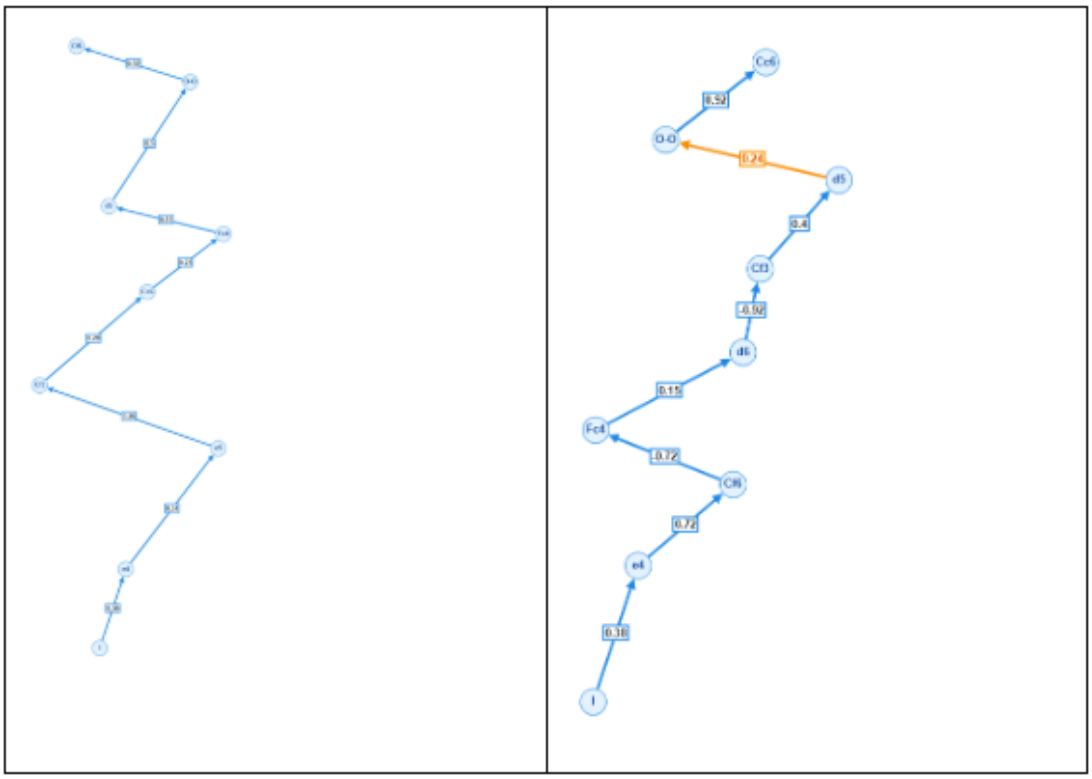


FIGURE 5.4 – Deux graphes différents menant à une position finale similaire

L'exploration avec une mémorisation des positions déjà visitées permet une réduction stricte du nombre de sommets explorés par rapport à une arborescence.

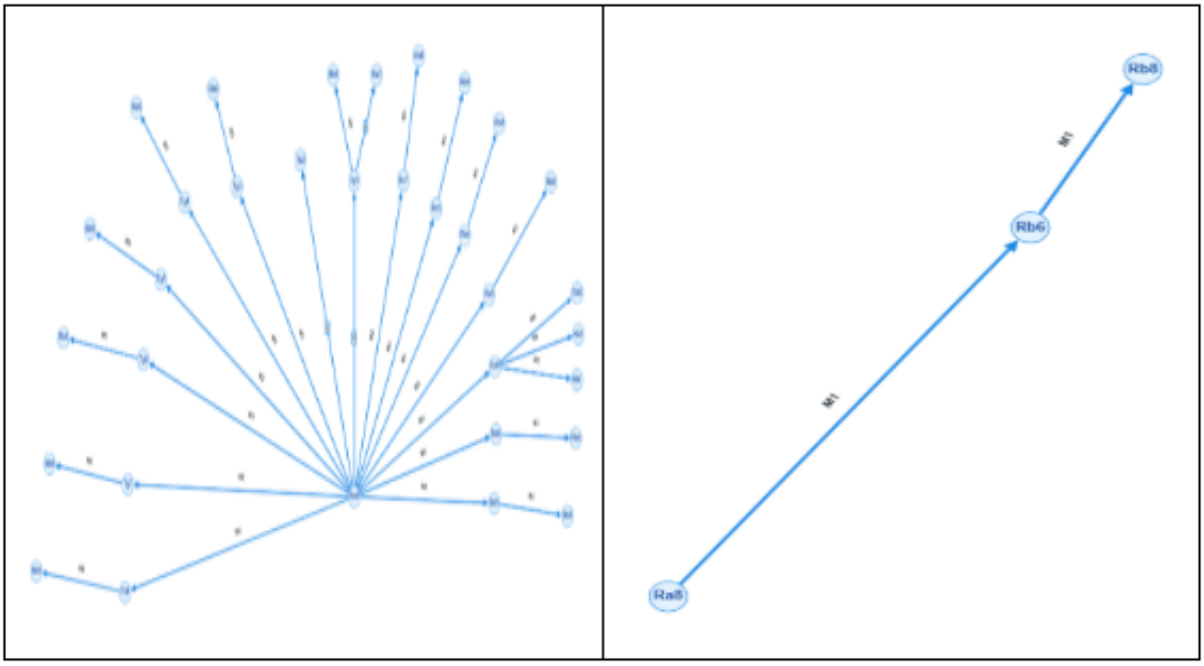
C'est une des raisons pour lesquelles on utilise l'élagage beta-alpha pour réduire la complexité de recherche dans le graphe.

5.3 Élagage Alpha-Beta

L'élagage alpha-beta est une méthode de simplification de graphe dans l'utilisation d'un algorithme minimax. Elle permet de réduire le nombre de calcul de certains "sous-arbres" dès lors qu'on est sûr qu'ils seront inutiles pour la suite de l'algo minimax.

On transforme ainsi le graphe initialement complet pat un graphe partiellement exploré où certains sous-graphes sont coupés.

Exemple : un exemple assez frappant, reprenons la finale vue plus haut. Le graphe initial devient :



En ne gardant que la valeur d'utilité optimale dans le graphe initial, on lit la suite de coups optimaux : $Rb6 \Rightarrow Rb8$ (puis mat en 1)

L'élagage alpha beta va supprimer toutes les valeurs d'utilité moins optimale que celle-ci, ici, le meilleur coup est Rb6 et est unique dans la finale, voilà pourquoi toutes les autres issues sont ignorées.

Vocabulaire :

α : borne inférieure, meilleur score garanti pour Max jusqu'ici

β : borne supérieure : meilleur score garanti pour Min jusqu'ici

Naturellement on, a la proposition suivante : $\alpha \leq minimax(n) \leq \beta$

On effectue une coupure lorsqu'on cesse d'explorer les frères d'un noeud n car on a prouvé qu'ils ne peuvent pas améliorer la valeur du parent.

Il existe deux types de coupures :

— Coupure α : Min a trouvé un coup si mauvais que Max ne choisira jamais cette branche

— Coupure β : Max a trouvé un coup si bon que Min ne le laissera jamais se produire

Dans l'exemple précédent, on a effectué 14 coupures β d'où le graphe résultat.

Cette méthode d'optimisation permet de réduire considérablement la complexité d'un algorithme : on pose T, la complexité en temps de l'algorithme et b étant le nombre de coups moyens par positions.

Algo minimax classique :

$$T = \Sigma(i = 0 \text{ à } d)b^i = (b^{(d+1)} - 1)/(b - 1) = O(b^d)$$

Algo minimax avec élagage optimal :

$$T' = O(b^{(d/2)}) \text{ complexité réduite considérablement}$$

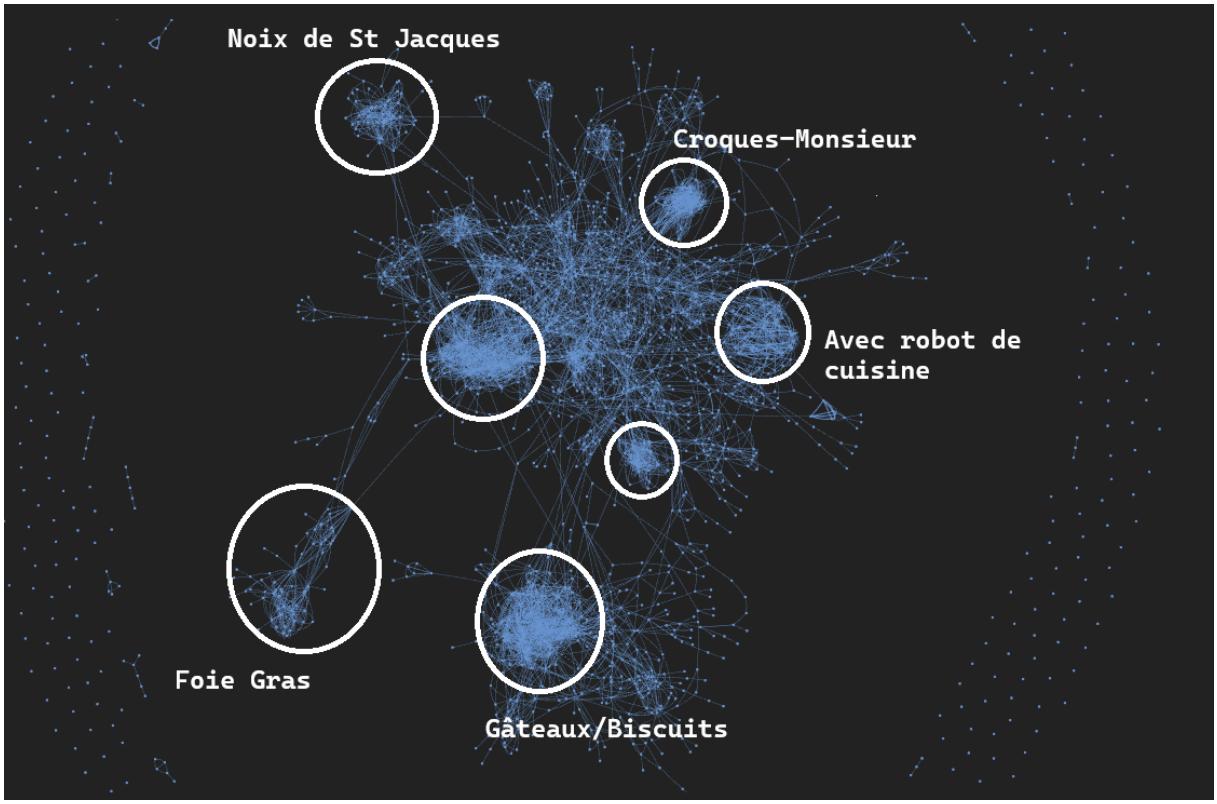
Conclusion

Nous avons pu avoir un aperçu de ce qu'est la théorie des graphes et quelques applications. Cependant, c'est un domaine si vaste qu'une vie entière ne suffirait pas à en faire le tour ! Avec les avancées des technologies informatiques, de nombreuses pistes de recherches et nouvelles manières d'utiliser les graphes font leurs apparitions. Vous serez donc probablement amenés à en entendre parler à nouveau.

Finissons ce tour d'horizon des applications de la théorie de graphes avec un dernier exemple réalisé par mes soins lors d'un projet d'analyse de données. L'idée était de mettre en lumière des correspondances entre différents plats, à partir uniquement de leurs recettes.

Un des moyens utilisé fût un graphe pondéré : un graphe dont chaque arête possède un poids. A l'aide d'un indice de similitude entre 0 et 1, qui décrit la correspondance entre 2 recettes, j'ai tracé un graphe à 2000 sommets, chacun représentant une recette, et la taille des arêtes représentant la similarité.

Cette représentation d'une matrice de similarité (de taille 2000 par 2000, donc totalement illisible) permet une bien meilleure visualisation des données, on est capable de distinguer des groupes qui se forment.



Bibliographie

[1] https://fr.wikipedia.org/wiki/Leonhard_Euler

[2] https://www.imo.universite-paris-saclay.fr/~ruette/mathsdiscretes/polygraph-Sigward.pdf

[3] Frédéric Havet, Adrian Bondy, Uppaluri Siva Ramachandra Murty. Théorie des graphes. 2025. ffhal05211979f

[4] https://www.gerad.ca/Sebastien.Le.Digabel/MTH8415/9_Graphes1.pdf

[5] https://people.irisa.fr/Francois.Schwarzentruber/algo1/09flots.pdf

[6] https://pagesperso.g-scop.grenoble-inp.fr/~stehlikm/teaching/inf303/5-plus-court-chemin.pdf

[7] https://cp-algorithms.com/graph/edmonds_karp.html

[8] https://youtu.be/qNqKGTtkwGw?si=mIHlQZxhT3ZxdQ2Y

[9] Cours d'Algorithmique de graphes de Frantisek Kardos

[10] https://major-prepa.com/mathematiques/probleme-sept-ponts-konigsberg/

[11] https://fr.wikipedia.org/wiki/Graphe_eul%C3%A9rien

[12] https://www.bibmath.net/dico/index.php?action=affiche&quoi=./b/biparti.html

[13] https://homepages.cwi.nl/~lex/files/histtrpclean.pdf

[14] Cours d'algorithmique pour les fonctions minimax et d'élagage

[15] https://www.youtube.com/watch?v=f30Ry1WOe_Q

[16] https://www.techno-science.net/glossaire-definition/Theoreme-du-minimax-de-von-Neumann.html

[17] https://www.youtube.com/watch?v=0qyqaK67Q50

[18] https://fr.wikipedia.org/wiki/Algorithme_minimax

[19] https://www.youtube.com/watch?v=Yo5vRHJ5y0Y&t=2172s

[20] https://graphonline.top/fr/

[21] https://www.chess.com/classroom

[22] https://github.com/louisbrulenaudet/latex-modele-memoire

[23] https://www.cpt.univ-mrs.fr/~masson/Formation-CNRS-LaTeX-Doc/Pegourie-Gonnard%20-%20Aide-memoire%20LaTeX.pdf